

AI agent for smart plug management: Model Context Protocol approach

Rares Vasiliu

May 2026

1 Introduction

The project focuses on developing a system that enables users to manage a home automation setup with smart plugs through natural language interactions in a chat interface. The goal is to create an intelligent agent that assists users in operating and managing their system, supporting core functionalities and executing predefined requests within its designed capabilities.

To achieve this system will be developed using a promising technology for developing ai agents which is MCP that should faster and make easier the development of AI agents.

2 Requirement analysis

2.1 Smart plug environment

The smart plug environment allows users to create, update, and delete smart plugs within their home. It also enables users to control each plug's status (on/off), indicating whether it is currently consuming power. Each smart plug stores a set of information, including its type and a fixed consumption value, which can be queried to monitor overall household energy usage.

2.2 Use Cases

- **Control Smart Plug State:** The user can turn an individual smart plug on or off.
 - *Task:* Interpret the user's request, identify the target plug, and update its status accordingly.
 - *Agent response:*
 - * The kitchen plug has been turned on.
 - * I've switched off the bedroom lamp plug.

- **Query System State:** The user can query the system to monitor the current situation (e.g., which plugs are on, overall consumption).
 - *Task:* Retrieve and summarize relevant information about the smart plugs, such as their status (on/off) and energy consumption.
 - *Agent response:*
 - * The following plugs are currently on: Kitchen Plug, Desk Lamp.
 - * The total energy consumption is 25W. The active plugs are: TV Plug and Heater Plug.
- **Create a New Smart Plug:** The user can create a new smart plug. The system must request confirmation before completing the action.
 - *Task:* Collect the required information (e.g., name, type, consumption), ask for confirmation, and create the plug upon approval.
 - *Agent response:*
 - * You are about to create a new smart plug named “Desk Lamp” with a consumption of 10W. Do you confirm?
 - * The new smart plug “Desk Lamp” has been successfully created.
- **Delete an Existing Smart Plug:** The user can delete an existing smart plug. The system must request confirmation before completing the action.
 - *Task:* Identify the plug to delete, request confirmation, and remove it upon approval.
 - *Agent response:*
 - * You are about to delete the smart plug “TV Plug”. Do you confirm?
 - * The smart plug “TV Plug” has been successfully deleted.
- **Handle Ambiguous Requests:** If a user request is ambiguous or incomplete, the system must ask for clarification.
 - *Task:* Detect missing or unclear information and prompt the user for the required details.
 - *Agent response:*
 - * I’m not sure which plug you mean. Could you specify the name or location?
 - * Do you want to turn the plug on or off?
- **Handle Out-of-Domain Requests:** If a request does not belong to the system’s domain, the agent must refuse it and avoid generating incorrect or fabricated information.
 - *Task:* Classify the request as out-of-scope and respond with a clear refusal.

– *Agent response:*

- * I'm sorry, but I can only help with managing smart plugs in your home.
- * That request is outside my capabilities. I can assist with controlling or monitoring your smart plugs instead.

2.3 Model Context Protocol

2.3.1 What is MCP

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems. Developed by Anthropic, it aims to standardize how large language models, such as Claude (also developed by Anthropic), acquire contextual information and interact with tools, APIs, and data sources.

Why does MCP matter?

The benefits of MCP are several, the main one are:

- LLM knowledge is limited on the information included in their training data. MCP enables them to access to external data, enhancing their capabilities and improving the overall end-user experience.
- MCP is designed to be plug-and-play, simplifying the integration of external tools and services. This reduces both development time when building AI applications or agents.

Client-Server model

MCP is based on client server model:

- **MCP Host:** is the main application, responsible for managing communication with the LLM.
- **MCP server:** provides tools and data access capabilities. Each MCP Server provides a set of specific tools responsible for retrieving information from local data or remote services.
- **MCP Client:** maintains a connection to an MCP server and obtains context from an MCP server for the MCP host to use. Each host can have multiple MCP client.

2.4 From LLM to language agents

Language models are stateless, meaning they do not persist information across calls. To overcome this limitation, LLMs are often embedded within *language agents*, which extend their capabilities by introducing memory, planning, and interaction with external environments. In this architecture, the LLM supports reasoning and decision-making.

A formalization of this idea is provided by the CoALA framework (Cognitive Architectures for Language Agents), which models language agents as cognitive systems composed of several components for perception, memory, reasoning, planning, and action.

To address the limitation mentioned previously, CoALA architecture model proposes several memories:

- **Working memory:** maintains active and readily available information as symbolic variables for the current decision cycle. This includes perceptual inputs, active knowledge (generated by reasoning or retrieved from memories), and other core information carried over from the previous decision cycle. On each LLM call, the LLM input is synthesized from a subset of working memory (e.g., a prompt template and relevant variables). The LLM output is then parsed back into other variables (e.g., an action name and arguments) which are stored back in working memory and used to execute the corresponding action.
- **Episodic and semantic memories:** are the declarative memories of the architecture that store experience and agent’s knowledge about the world and itself. During the planning stage of a decision cycle, these episodes may be retrieved into working memory to support reasoning. An agent can also write new experiences from working to episodic memory as a form of learning. Semantic memory, is retrieved for reasoning or decision-making.
- **Procedural memories:** stores the knowledge about when and how to act. This includes the representations of actions, procedures, or strategies that the agent can execute to achieve its goals.

Using MCP, the management of the procedural memory and LLM interactions becomes much simpler. The procedural memory is represented by the set of tools in the MCP servers and additional rules to manage them. When the model supports tool calling, the LLM output already includes the list of tool invocations to execute, with the correct tool names and parameters generated in a more precise way in a dedicated generation step. This approach is much simpler compared to act-only prompting or ReAct (Reasoning and Acting) pattern where the name and parameters are part of the text response and are more susceptible to hallucination.

2.5 Prompting method

The behavior and the reasoning capabilities of the agent depend on the prompt paradigm used to guide the language model. Since this project requires the agent not only to answer questions, but also to reason about user intentions, interact with tools and manage dialogue state simpler prompting strategies such as *Direct/Standard Prompting* and *Chain-of-Thought (CoT) Prompting* were considered insufficient:

- Direct prompting follows a standard zero-shot paradigm in which the language model directly answers a question or solves a task without explicitly exposing intermediate reasoning steps.
- CoT prompting improves reasoning by encouraging the model to “think step by step” before producing an answer. Although CoT enhances the model’s reasoning capabilities, it mainly focuses on internal reasoning and does not explicitly structure the interaction with the environment.

Another alternative is *Act-only prompting*, where the model focuses primarily on selecting actions or tool calls but without explicitly modeling reasoning. Although this strategy can work well for deterministic command execution, it presents several limitations in conversational environments. In particular, the absence of explicit reasoning makes ambiguity handling more difficult. For example, if multiple smart plugs match the same description, or if required information is missing, the agent may struggle to determine the correct next step or request the appropriate clarification from the user.

2.5.1 ReAct Prompting

To address the limitations of the previous approaches, this project adopts the ReAct (Reason + Act) prompting technique. ReAct combines explicit reasoning steps (Thoughts), with executable operations (Actions).

The agent alternates between internal reasoning and actions with either external tools or the user. The external tools provide observation to the agent, which represents the result returned after executing an action. These observations are then used in the next reasoning cycle to guide subsequent decisions.

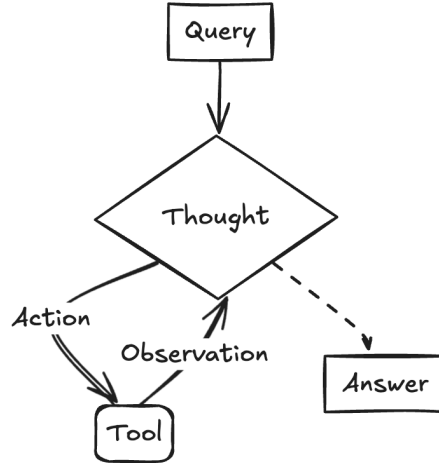


Figure 1: ReAct decision cycle

An additional important aspect of the adopted approach is the use of *Few-shot In-Context Learning*. LLMs in ReAct are prompted with few-shot in-context examples to generate both domain-specific actions and free-form language thoughts for task solving. In-context learning is a paradigm that allows language models to learn tasks given only a few examples in the form of demonstrations.

3 Implementation

3.1 Smart plug service

The smart plug service is a lightweight microservice responsible for managing smart plug devices within the system. Its primary purpose is to simulate and expose the state and energy consumption of household appliances through a simple RESTful API.

Each smart plug is represented by a data model containing a unique identifier, a human-readable name, the type of utility it consumes (e.g., electricity), its current status (on/off), and its real-time consumption value.

The service is implemented using **FastAPI** in python. The REST API exposes the following endpoints:

- GET `/api/smart-plugs` – Retrieve all smart plugs
- POST `/api/smart-plugs` – Create a new smart plug
- PUT `/api/smart-plugs/{plug_id}` – Update an existing smart plug
- PATCH `/api/smart-plugs/{plug_id}` – Toggle the status (on/off) of a smart plug
- DELETE `/api/smart-plugs/{plug_id}` – Remove a smart plug

3.2 Smart Plug MPC server

The MCP server is build using **FastMCP**, a framework for building MCP applications. The MCP server acts as a lightweight wrapper around the smart plug microservice REST API. This layer enables LLM-based applications to interact with smart plugs using standardized tool calls.

3.2.1 Tools

The exposed tools include:

- **get_all_plugs**: retrieves all smart plugs
- **create_plug**: creates a new smart plug
- **update_plug**: updates existing plug attributes

- **switch_plug**: toggles the plug state (ON/OFF)
- **delete_plug**: removes a smart plug

The tool name is compute from the function name, while the tool description is given from the function's docstring, for example:

```
@mcp.tool()
async def get_plug(plug_id: str) -> dict:
    """
    Return a smart plug by id.

    Args:
    - plug_id: unique identifier

    Output:
    - SmartPlug object

    Error:
    - 404 if not found
    """
    async with _client() as client:
        ...
```

3.2.2 API communication

Each tool internally uses an asynchronous HTTP client to communicate with the smart plug service running locally.

Each tool performs basic error handling by interpreting HTTP status codes and returning structured responses suitable for LLM consumption. This design keeps the MCP layer minimal, acting primarily as an adaptation layer between the LLM and the underlying RESTful smart plug service.

3.3 Agent architecture

Combining the MCP element for tooling and CoALA elements for context management we obtain the architecture in figure 2:

- **User chat**: the entry point where the user interacts with the agent.
- **Message analysis**: analyzes the user message (either a new task or a continuation of the current task) and prepares or updates the prompt elements.
- **Reasoner**: the decision-making engine that performs the ReAct reasoning cycles.
- **LLM**: large language model client.

- **Prompt builder:** centralizes prompt engineering by generating the different types of prompts, such as system prompts, user prompts, and working memory prompts.
- **Working memory:** stores the temporary reasoning state.
- **Procedural memory:** stores behavioral rules and execution policies.
- **Episodic memory:** stores few-shot in-context learning examples in the form of episodes.
- **Semantic memory:** stores factual knowledge.
- **Smart plug MCP client, server, and service:** components for tool interaction and communication.

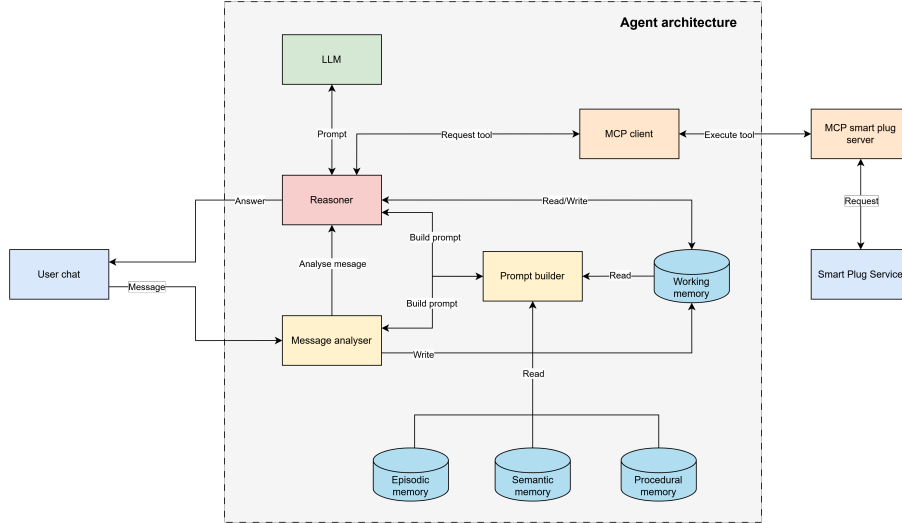


Figure 2: Smart plug agent architecture. In light blue the environment components, in azure the memory components, in orange the MCP components, in yellow the agent functional components, in red the reasoning cycle component, and in green the LLM component.

3.4 Memory Implementation

The agent adopts a CoALA memory components, composed of four different memory systems: working memory, semantic memory, procedural memory, and episodic memory. Each memory has a specific role in supporting reasoning, tool usage, and context management.

3.4.1 Working Memory

The working memory stores the temporary reasoning state of the agent during task execution. It is implemented as a key-value store for general temporal information combined with a reasoning history buffer.

The **store** dictionary maintains the current execution context, including system prompts, task prompts, token statistics, reasoning cycle counters. The **history** list stores intermediate reasoning traces, tool observations, and generated thoughts. This history is continuously updated during the ReAct loop and injected back into the prompt to preserve short-term contextual awareness.

The memory is reset after task completion to avoid contamination between independent tasks.

3.4.2 Semantic Memory

Semantic memory stores persistent factual knowledge about the smart plug domain. The knowledge is represented as a list of textual facts that are directly injected into the system prompt.

Examples of stored knowledge include:

- The meaning of real-time consumption.
- Supported measurement units.
- The semantic meaning of the plug status.

This memory provides the agent with domain-specific information required for reasoning and answering user queries consistently.

3.4.3 Procedural Memory

Procedural memory stores behavioral rules and execution policies that regulate the agent’s reasoning process and tool usage.

The rules are represented as a list of textual instructions injected into the prompt. These rules define:

- The required reasoning format.
- How tools should be invoked.
- Permission handling policies.
- Safety constraints.
- Response formatting requirements.
- Optimization constraints such as minimizing token usage.

3.4.4 Episodic Memory

Episodic memory stores few-shot in-context learning examples as structured episodes loaded from a JSON file.

Each episode contains task type, user task, and a complete reasoning trace. The reasoning trace follows the ReAct paradigm and includes: intermediate thoughts, tool calls, observations, and final answers.

These examples guide the language model by demonstrating correct reasoning patterns, tool usage behavior, and response formatting strategies for similar future tasks.

4 Evaluation

The evaluation focuses on assessing the correctness of the agent’s behavior and responses across different task categories derived from the considered use cases. In particular, the evaluation analyzes whether the agent is able to select the appropriate tools, follow the expected reasoning procedure, and generate correct final answers without violating the defined behavioral constraints.

4.1 Query-response tasks

This category evaluates the agent’s ability to correctly answer information retrieval and monitoring queries. Three types of queries are evaluated:

1. *Plug information*: e.g. "Tell me information about the water plugs in my house".
2. *Monitoring*: e.g. "Tell me how much gas plugs are consuming right now".
3. *Plug status*: e.g. "Tell me how which plug is on"

For these tasks, the expected behavior is for the agent to retrieve the required information using the `get_plugs` tool, analyze the response, and generate a correct final response.

The evaluation also verifies that the agent follows the expected execution policy. In particular, the agent should:

- Avoid requesting user permission, since the tasks are non-critical.
- Use only the required plug retrieval tool.

During the evaluation procedure, the agent is prompted with a task and its generated response is then analyzed.

1. **Correct**: correctness is determined by checking the presence of a predefined set of expected keywords within the final answer. If all expected keywords are present, the response is marked as correct.
2. **Fail for Answer**: the agent produces an answer but is incorrect or incomplete.

3. **Fail for Ask:** the agent unnecessarily requests user permission.
4. **Fail for Error:** the agent fails to produce a final answer, e.g. by running out of reasoning cycles.

4.2 Create plug tasks

This category evaluates the agent’s ability to correctly manage protected actions requiring explicit user authorization. In particular, the evaluation focuses on the creation of new plugs through the `create_plug` tool.

Two scenarios are evaluated:

1. *Ask for plug creation without giving permission:* the user asks to create a new plug but does not provide permission.
2. *Ask for plug creation giving permission:* the user asks to create a new plug and subsequently provides permission.

For these tasks, the expected behavior is for the agent to request user confirmation before executing the protected action. Only after receiving a positive permission response should the agent invoke the `create_plug` tool.

The evaluation also verifies that the agent follows the expected execution policy. In particular, the agent should:

- Always request user permission before calling the `create_plug` tool.
- Avoid executing the tool if permission is denied or not provided.
- Execute the tool only after receiving explicit user confirmation.

During the evaluation procedure, the agent is prompted with a task and its generated responses are analyzed across the interaction steps. Responses are classified as follows:

1. **Correct:** the agent correctly requests permission and behaves consistently with the provided user response.
2. **Fail for Answer:** the agent executes the protected action or generates a final answer without first requesting permission.
3. **Fail for Insist:** the agent unnecessarily requests permission multiple times after already receiving a permission response.
4. **Fail for Tool Usage:** the agent incorrectly calls the `create_plug` tool when permission is denied, or fails to call it after permission is granted.
5. **Fail for Error:** the agent fails to produce a final response, e.g. by running out of reasoning cycles.

For allowing creation the responses are classified as follows:

1. **Correct:** the agent correctly requests permission and behaves consistently with the provided user response.
2. **Fail for Answer:** the agent produces an incorrect answer.
3. **Fail for Insist:** the agent unnecessarily requests permission multiple times after already receiving a permission response.
4. **Fail for Tool Usage:** the agent calls the `create_plugin` tool multiple times and produces a wrong answer.
5. **Fail for Error:** the agent fails to produce a final response.

5 Results and Observations

The table below presents the results of the evaluation discussed in the previous chapter. For each task, several evaluation cycles were conducted, each consisting of 10 iterations and the averaged value across these runs was taken. For the evaluation the model used were: gpt-oss:20b-cloud and gpt-oss:120b-cloud using Ollama as client. These models are OpenAI’s open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases. The metrics reported in the results describe the system’s behavioral performance using the percentage of correct and incorrect responses, the number of tool calls per task, the number of reasoning cycles, and token usage for both input and output. For token usage, both mean and median values are provided.

Task	Model	S	F_m	F_a	F_t	E	$\#T_{pt}$	Avg Steps	Avg T_{in}	Median T_{in}	Avg T_{out}	Median T_{out}
Status Query	20B	75%	25%	0%	–	0%	1.2	3.0	6508.4	6353.0	810.6	624.5
	120B	100%	0%	0%	–	0%	1.0	2.6	5361.8	6239.5	624.5	631.5
Info Query	20B	95%	5%	0%	–	0%	1.4	3.4	9228.1	6213.0	625.6	617.5
	120B	95%	5%	0%	–	0%	0.9	2.8	5369.9	4606.5	382.2	368.5
Monitoring Query	20B	95%	5%	0%	–	0%	1.3	2.6	5564.4	5285.0	460.0	370.5
	120B	95%	5%	0%	–	0%	0.9	2.2	4250.0	3825.5	297.9	307.5
Deny Permission	20B	30%	10%	0%	60%	0%	1.3	3.9	7334.5	5505.5	970.6	1014.0
	120B	100%	0%	0%	0%	0%	1.0	2.1	3471.0	3289.5	514.9	476.0
Allow Permission	20B	80%	0%	0%	10%	10%	3.2	6.2	12573.4	1055	1449.4	1245
	120B	90%	0%	0%	0%	10%	1.2	3.7	7920.7	6244.5	735.3	630

Table 1: summarizes the evaluation results across different task categories for the two tested models (gpt-oss:20b-cloud and gpt-oss:120b-cloud). S denotes success rate; F_m , F_a , and F_t represent failures due to wrong answers, permission-related issues, and tool-use errors respectively; E denotes generic errors. $\#T_{pt}$ indicates the average number of tool calls per task, and Avg Steps the average number of reasoning steps. T_{in} and T_{out} refer to input and output token counts, reported as both mean and median.

5.1 Observations

5.1.1 Query tasks

For the query task, we observe that there are no execution errors across either model, indicating that both systems consistently provide an answer and do not get stuck in invalid decision loops. However, the failure-to-match cases are not due to incomplete responses, but rather due to the agent failing to properly invoke tools to retrieve the required data. This pattern is highlighted in the 120B model, since it's more efficient with tool usage overall and avoids redundant tool calls compared to the 20B model, we see < 1 tool calls per task which means that it still occasionally avoid using tools at all. Both models have the same problem, the agent tends to terminate its reasoning process too early, leading to premature answers instead of fully leveraging available tools to resolve the query.

5.1.2 Permission handling

The main limitation of gpt-oss:20b appears in permission handling. When the model is prompted to directly add a plug, it tends to interpret this instruction as implicit user authorization to proceed, rather than recognizing the need for an explicit permission step, leads to the tool execution. This behavior is shown also with episodic memory: when a positive permission example is included in memory, the agent may incorrectly generalize it as an active user approval for the current interaction. This can cause it to bypass the intended protection mechanisms and execute actions without re-confirmation.

A further weakness lies in how permissions are positioned within the interaction history. If a permission message appears embedded as observation, the agent may treat it as outdated and request confirmation again unnecessarily, leading to redundant permission loops. A more reliable setup is to place the permission explicitly after the user's current task, ensuring it is correctly associated with the active request.

For the first issue, however, there is no direct fix. Attempts to mitigate it, such as introducing explicit planning constraints in the system rules and enforcing permission as a required parameter in MCP tool calls, only partially improve behavior. These measures reduce failures from 100% to around 60%, which remains too high.

The correctness of permission handling is seen when testing ambiguous request task such as when non-existent plug is queried. In both 20B and 120B models, the agents correctly avoid using the create-plug tool. When the reasoning process leads toward a creation action, both models appropriately request user confirmation first, showing that the permission logic works, even if it remains fragile in edge cases.

When permission is granted to the agent, a major issue is that the create action is often executed twice in the 20B model. With this behavior the model is sometimes driven to assume that the plug already exists, leading to redundant creation attempts. The same behavior can be observed when attempting to add

an already existing plug. The agent tries to add it again and produces a positive response.

6 Conclusion and further work

This project shows that Model Context Protocol (MCP) can simplify the development of tool-using AI agents. By separating the smart plug service from the agent layer, MCP allows developers to focus on reasoning logic rather than the boilerplate of API integration. At the same time, the evaluations highlight an important limitation of relying too heavily on the language model for autonomous decision-making. While the agent performs well in many query tasks, protected actions such as creating or deleting plugs require stricter control. In these cases, the model may overgeneralize, misinterpret user intent, or repeat actions unnecessarily. This confirms that for safety-critical or state-changing operations, LLM reasoning should be complemented by explicit application guards.

Another relevant observation is that tool access alone is not sufficient for building reliable autonomous agents. The behavior of the system depends not only on the quality of the language model, but also on how reasoning, memory, and interaction policies are organized around it. For this reason, the project combines MCP with concepts inspired by the CoALA cognitive architecture. The adoption of CoALA-inspired memory components proved particularly valuable for structuring both the agent behavior and the underlying code.

6.1 Learning value of the project

- Practical exploration of MCP as an emerging standard for AI tool integration.
- Studying CoALA to investigate the role of different memory types in supporting reasoning and context management.
- The combination of MCP and CoALA demonstrates that modern language agents require more than simple prompt-response interactions. MCP provides the procedural interface to the environment through tools, while the memory architecture provides contextual continuity and behavioral consistency.
- Using ReAct to combine reasoning and tool execution in iterative decision cycles provides practical experience with LLM-based agents and shows the importance of reasoning in decision-making systems that use tools.
- From a software engineering perspective, the project demonstrates how to design and implement an AI agent capable of reasoning and acting.

6.2 Future work

- One of the main limitations of the current implementation is the design of the working memory. At the moment, the working memory is focused primarily on supporting the current active task and maintaining temporary reasoning state during the ReAct cycle. While this is sufficient for short interactions, it becomes less effective during longer multi-turn conversations.
- Implement hard-coded guardrails: rather than asking the LLM to remember to ask for permission, the agent framework should intercept protected tool calls and force a UI-level confirmation (e.g. with buttons) that the LLM cannot bypass.

Appendix

A Reasoning step example

Context Window

```
system: "You are a smart home assistant that manages and monitor user
smart plugs. Don't answer question out of this domain.\nYou always
follow the Thought, Action format. Never respond with plain text
only.\n\nIn the CHAT HISTORY you can find previous Thought, Action
cycles with Observations of action response to resolve the user
question. \nIf is empty use tools to make request. Before calling tools
check if the data is already there.\n==\nHOW and WHEN to ACT:\n1. How
to reason on the task: Use 'Thought:' to reason first about the current
task then on the content of CHAT HISTORY and finally on the given
tools. Keep it concise and focused.How to respond to te user: Use
'Action[Answer]:' the content will be read by the user user. This must
be clear and user-friendly.How to use CRITICAL tools: If a tool is
marked with 'CRITICAL: Requires user confirmation.', you MUST STOP and:
first explain your intention and expected outcome to the user to
receive the permission using 'Action[Permission]:'. Wait for user
confirmation in the next Observation formatted as:
UserPermissionResponse[<message>]. If the permission response is
positive, proceed with the tool call. If the permission response is
negative, do NOT call the tool and instead respond using
'Action[Answer]:'.Create plug is a CRITICAL tool, use only when you
have permission.\n2. How to get permission: if and ONLY if there is not
tag UserPermissionResponse in the CHAT HISTORY ask the user permission
with Action[Permission]. In this step dont generate any tool call.Only
Critical tools needs permission.\n3. Don't answer with incomplete
responses, don't tell the user technical information such as what tool
you are using.Read from the CHAT HISTORY the tool responses before
calling a tool, Never call a tool twice if is not necessary.To create a
plug verify you have from the user all necessary data to make the
request!\n4. Keep the answer as short as possible (TO SAVE TOKENS)\n5.
To calculate the real time consumptions consider only plugs with status
ONTo before switching a plug, check it's status. After switching check
the response in the CHAT HISTORY\n===\nFor any doubt use this
information:\n1. real_time_consumption -> how much does a plug
consumes\n2. measurement units -> ELECTRICITY (kWh) | WATER (m^3) | GAS
(m^3)\n3. status -> if the plug is consuming in real-time\n=="

user: What are the plugs in my house?

assistant: CHAT HISTORY - List of interactions:\nEmpty. Reason on the
current task to add new elements.
```

Agent response

```
content: We need to respond with list of plugs. We need to retrieve
plugins using get_all_plugs function. Not critical. So we can call it.
tool_calls: [ToolCall(function=Function(name='get_all_plugs',
arguments={}))]
```

B Interactions

SmartPlugAgent ready. Type 'exit' to quit.

You: What are the plugs in my house?

Agent [answer]: Here are the smart plugs currently set up in your home:

****Electricity****

- Refrigerator { ****ON****, 0.4 kWh
- Electric Oven { ****ON****, 1.5 kWh
- Washing Machine (Electric) { ****ON****, 1.0 kWh
- Electric Water Heater { ****ON****, 1.6 kWh
- Air Conditioner { ****ON****, 1.2 kWh
- Living-Room Lamp { ****ON****, 2.4 kWh

****Gas****

- Gas Boiler { ****ON****, 0.6 m³
- Gas Stove { ****OFF****, 0.35 m³
- Gas Water Heater { ****ON****, 1.2 m³
- Gas Fireplace { ****ON****, 0.8 m³
- Gas Dryer { ****ON****, 1.5 m³

****Water****

- Shower { ****OFF****, 3.0 m³
- Kitchen Faucet { ****OFF****, 1.7 m³
- Dishwasher { ****ON****, 2.4 m³
- Washing Machine (Water) { ****ON****, 0.5 m³
- Garden Sprinkler { ****OFF****, 1.5 m³

Let me know if you'd like details about any specific plug or want to change its status!

You: Tell me which plug are on

Agent [answer]: The following plugs are currently ON:

- Refrigerator (Electricity) { 0.4 kWh
- Electric Oven (Electricity) { 1.5 kWh
- Washing Machine (Electric) { 1.0 kWh
- Electric Water Heater (Electricity) { 1.6 kWh
- Air Conditioner (Electricity) { 1.2 kWh
- Gas Boiler (Gas) { 0.6 m³
- Gas Water Heater (Gas) { 1.2 m³

- Gas Fireplace (Gas) { 0.8 m³
- Gas Dryer (Gas) { 1.5 m³
- Dishwasher (Water) { 2.4 m³
- Washing Machine (Water) { 0.5 m³
- Living-Room Lamp (Electricity) { 2.4 kWh

You: can you turn on the dishwasher?

Agent [answer]: The dishwasher is already turned **on**. No further action is needed.

You: can you turn off the dishwasher?

Agent [answer]: **The dishwasher has been turned off successfully.**

You: I've bought a new Lamp, can you add it on the system?

Agent [ask]: I can add your new Lamp as a smart plug in the system. To do so, I need to create a plug entry with its name, the utility type (e.g., electricity), and an initial status (off) and consumption (0). Please confirm that I may proceed.

You: Yes! Call it Lamp, with 2.4 consumption. Currently is off.

Agent [answer]: Your new lamp has been added as a smart plug (ID: **lamp**). It's currently turned **off** and ready to be switched on whenever you need. Let me know if you'd like to turn it on or manage any other devices!